

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3067 – Redes de Computadoras

Sección 10

Ing. Kevin Velásquez



Excelencia que trasciende

DEL VALLE
GRUPO EDUCATIVO

Proyecto – ChatBot

Dulce Ambrosio (231143)

Guatemala, 21 de septiembre de 2026

Resumen

Los modelos grandes de lenguaje (Large Language Models, LLM) pueden interpretar lenguaje natural y conservar contexto, pero no consultan por sí mismos un inventario actual, ni ejecutan acciones sobre sistemas externos. Pharmacy MCP aborda esta separación por medio del Model Context Protocol (MCP), este modelo decide cuándo necesita una herramienta, el anfitrión controla su ejecución y los servidores publican contratos de manera independiente del proveedor de IA.

Este proyecto implementó de manera manual JSON-RPC 2.0 y el subconjunto necesario de MCP, sin FastMCP ni SDK de MCP, se realizó un servidor farmacéutico con siete herramientas para evaluar síntomas por medio de reglas controladas, consultar un catálogo simulado, revisar interacciones no exhaustivas, consultar un inventario y gestionar órdenes. Este mismo núcleo funciona de manera local por stdio y remotamente por medio de un subconjunto no streaming de Streamable HTTP. Un host propio lo que hace es integrar los servidores MCP oficiales de Git y Filesystem, descubre sus herramientas y les asigna namespaces reversibles.

Gemini se integró de manera directa con la API REST (generateContent), el host adapta historial, declaraciones de funciones, llamadas y resultados sin usar el SDK del proveedor. Se realizó una validación por medio de una conversación con contexto y por medio de la interfaz se añaden sesiones aisladas, estados de carga y errores y confirmaciones.

Introducción

Los LLM producen texto a partir de patrones aprendidos pero una respuesta del modelo no es equivalente a una lectura de estado actual, ni a una operación autorizada, para que el chatbot consulte un repositorio, un sistema de archivos o un inventario necesita herramientas externas, contratos de datos y una capa que controle su, cuando cada proveedor define su propio mecanismo, el mismo servicio debe adaptarse de manera repetida.

El MCP propone separar tres responsabilidades (el servidor, el cliente y el host), el servidor expone capacidades, el cliente mantiene una conexión y habla el protocolo, el host coordina clientes, contexto, modelo y políticas. El MCP utiliza JSON-RPC 2.0 en la capa de aplicación, así el servidor puede publicar herramientas de forma independiente del modelo que las solicita y del transporte que conecta ambos extremos.

Pharmacy MCP aplica esta arquitectura a una cadena de farmacias simulada, este casi permite estudiar consultas, mutaciones, persistencia, concurrencia, autorización, redacción de catálogo, inventario y reglas controladas.

Objetivos

Implementar y analizar un chatbot que utilice servidores MCP locales y remotos, construyendo de manera manual los mensajes JSON-RPC 2.0, el lifecycle, los transportes, la integración con un LLM y los controles de seguridad necesarios para un caso de uso farmacéutico simulado.

Alcance y Limitaciones

El catálogo, inventario, reglas de interacción, alergias, síntomas, recetas y órdenes son simulados, la evaluación de síntomas utiliza frases controladas y nunca delega una decisión clínica al LLM.

El estado local se almacena en SQLite, en un único filesystem puede persistir aperturas, pero en Cloud Run reside bajo /tmp y es efímero. El servicio se limita a un máximo de una instancia para reducir inconsistencias, aunque una instancia aún puede reiniciarse o ser reemplazada, las sesiones MCP y el historial del chatbot permanecen en memoria.

El transporte HTTP implementa respuestas JSON, tampoco admite batch JSON-RPC, documentos JSON multilínea en stdio, paginación de herramientas ni notificaciones de cambio del catálogo de tools. La interfaz web se sirve solo en loopback y no se despliega públicamente, Gemini está sujeto a disponibilidad de modelo, cuota y condiciones de la cuenta, la validación real no se considera una garantía futura.

Fundamentos Teóricos

JSON-RPC 2.0

Representa una invocación por medio de un objeto con jsonrpc, method, params e id. Una respuesta exitosa conserva el mismo id y contiene result, una respuesta fallida contiene error, una notificación no incluye el miembro id y por definición no

recibe una respuesta, en esta implementación, un id nulo sigue siendo una solicitud y recibe una respuesta con id: null.

Model Context Protocol

El MCP estandariza como un host descubre y usa capacidades externas, el lifecycle negocia la versión e identidad con initialize, se confirma con notifications/initialized y después habilita operaciones como tools/list y tools/call. Una tool describe nombre, propósito y schema de entrada, su resultado puede incluir texto y contenido estructurado, el protocolo no decide por sí mismo si una operación es segura, el host debe imponer políticas y pedir autorización cuando corresponde.

Host, Cliente y Servidor

El servidor ejecuta la capacidad, el cliente mantiene una sesión con ese servidor, el host coordina clientes, modelo, conversación, políticas y presentación. Pharmacy MCP mantiene la división de MCPServerManager (administra clientes) con ChatOrchestrator (coordina el LLM y las tools y cada servidor conserva su propio lifecycle).

Transportes stdio y Streamable HTTP

En stdio cada línea física contiene un objeto JSON completo codificado en UTF-8, stdout se reserva el protocolo y stderr a diagnósticos, en HTTP cada mensaje se

envía por POST/mcp, la inicialización crea una sesión opaca, los mensajes siguientes declaran la versión y el cierre usa DELETE/ mcp. Un GET/health independiente permite comprobar disponibilidad sin cerrar sesión.

Namespaces y Function Calling

Cuando varios servidores publican nombres iguales, el host genera un nombre global reversible con <servidor>_<tool>, al invocar se recupera el servidor y el nombre original. Para Gemini esas definiciones se convierten en function declarations, el modelo devuelve functionCall, el host ejecuta MCP y agrega functionResponse al historial antes de solicitar la respuesta final.

HTTP, TCP, IP y TLS

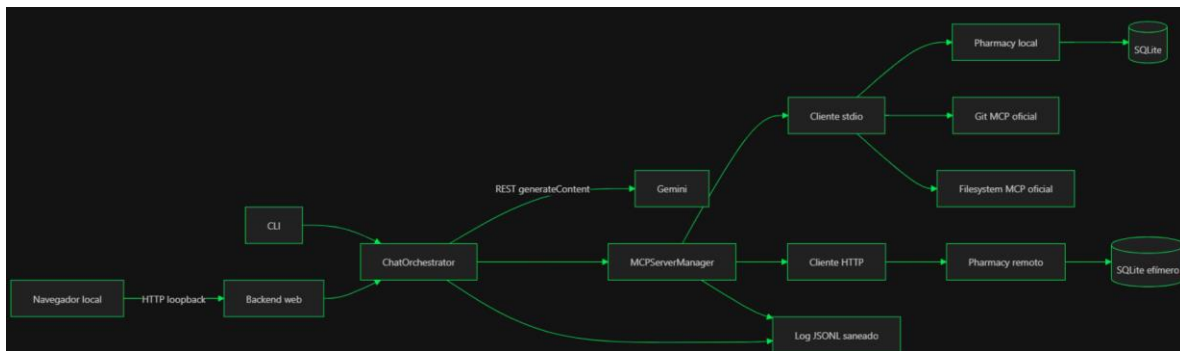
HTTP y MCP operan en la capa de aplicación, TCP ofrece un flujo confiable, orientado a conexión y ordenado por medio de números de secuencias, la IP encamina paquetes entre orígenes y destinos, la capa de enlace entrega tramas en el alcance local para el servidor remoto, TLS protege HTTP y Wireshark observa la negociación y registros cifrados, pero no el JSON-RPC sin claves de sesión o un mecanismo explícito de descifrado.

Arquitectura

El flujo local usa procesos hijos y NDJSON por stdio, el remoto usa HTTPS y una sesión Streamable HTTP. El navegador no conoce credenciales, por lo que habla con el backend local, que comparte el mismo orquestador y manager de la CLI.

En un turno con tools, el host envía historial y schemas a Gemini y si la respuesta contiene una llamada, valida nombre, identificador y argumentos, aplica la política, pide confirmación para mutaciones, llama al servidor, limita y normaliza el resultado y vuelve a consultar al modelo. Las llamadas se procesan secuencialmente y tienen límites de rondas, cantidad y tamaño.

Diagrama 1. Arquitectura lógica



Implementación manual de JSON-RPC y MCP

Las clases del paquete jsonrpc validan versión, método, parámetros, identificadores y exclusión mutua entre result y error. La serialización rechaza valores no representables y la deserialización rechaza JSON inválido, constantes no finitas y mensajes cuya forma no corresponde al protocolo. Los clientes incrementan IDs y correlacionan cada respuesta, las notificaciones se envían sin esperar una línea o cuerpo de respuesta.

El servidor transita por UNINITIALIZED, INITIALIZING y READY. Solo acepta la revisión 2025-11-25. Antes de READY, tools/list y tools/call devuelven -32002. Un segundo initialize es inválido. En stdio, el EOF constituye el cierre limpio, en HTTP, el cliente elimina la sesión con DELETE.

Se distingue un error de forma de un error de dominio. Un argumento ausente, tipo incorrecto o identificador mal formado produce JSON-RPC -32602. En cambio, una solicitud bien formada para un SKU inexistente, stock insuficiente o una orden desconocida produce una respuesta JSON-RPC exitosa cuyo resultado MCP contiene isError: true. Esa separación evita confundir fallas del contrato con

Los servidores Git y Filesystem son distribuciones oficiales externas, pero el cliente, framing, lifecycle, routing, logging y políticas que los integran fueron implementados en el proyecto.

Servidor Pharmacy MCP

Tool	Propósito	Argumentos		Salida	Errores Relevantes
		Principales			
assess_symptoms	Clasificar síntomas con reglas controlada	symptoms; age y duration_da	Severidad, categoría, coincidencias, banderas,	Forma inválida: -32602	

	s y priorizar banderas urgentes	ys opcionales	acción y disclaimer	
search_medications	Buscar catálogo por nombre, alias, ingrediente, categoría o SKU	query; otc_only opcional	Coincidencias y precios GTQ	Forma inválida: -32602; cero resultados es éxito
get_medication_details	Recuperar la ficha completa de un SKU	sku	Datos de catálogo, contraindicación simulada y precio	SKU bien formado inexistente: isError
check_interactions	Comparar un medicamento con	medication_ sku; listas opcionales	Alertas, severidad máxima y	SKU desconocido: isError; forma inválida: -32602

	lista actual y alergias simuladas		límites del análisis	
check_stock	Consultar existencia s en una o todas las sucursale s	sku; branch_id opcional	Cantidad y disponibilida d por sucursal	Sucursal/SKU inexistente: isError
create_order	Crear una orden y reservar stock de forma atómica	branch_id, items; prescription _id opcional	Snapshot de orden, totales y estado created	Stock/receta/do minio: isError; forma inválida: - 32602
get_order_status	Leer el snapshot inmutable de una orden simulada	order_id	Líneas, total, estado y fecha	ID inexistente: isError; forma inválida: -32602

El catálogo contiene sucursales y medicamentos validados, el inventario exige una combinación por sucursal y SKU. El dinero se guarda como centavos enteros y se serializa como texto decimal con moneda, evitando errores de float. `create_order` abre `BEGIN IMMEDIATE`, comprueba cada línea, decrementa con una condición `quantity >= requested`, inserta orden e ítems y confirma todo en una transacción. Cualquier excepción ejecuta `rollback`. De este modo no existe una orden parcial y dos escritores concurrentes no pueden reservar las mismas unidades.

Las referencias de receta solo prueban un patrón sintáctico. Los resultados repiten que son simulaciones y que una ausencia de alertas no demuestra seguridad. Ante una bandera urgente, la evaluación prioriza atención médica y no recomienda seleccionar o comprar un producto.

Servidores Oficiales Git y Filesystem

El host integra `mcp-server-git==2026.8.18` mediante `uvx` y `@modelcontextprotocol/server-filesystem@2026.8.31` mediante `npx`. Las versiones están fijadas, sus herramientas se descubren con `tools/list`, no mediante una lista de despacho codificada en el host.

Git exige que cada `repo_path` coincida exactamente con la raíz canónica configurada. Filesystem valida `path`, `paths`, `source` y `destination`, rechaza rutas relativas, siblings y escapes por enlaces. Una tool solo se trata como lectura cuando sus annotations lo indican de forma inequívoca.

Filesystem enumera la raíz, rechaza primero una escritura no autorizada, crea un README tras confirmación y lo vuelve a leer. Git inspecciona el estado, rechaza staging sin permiso, agrega el archivo tras autorización, muestra el diff y crea un commit. La identidad Git es local al repositorio desechable, no se modifica la identidad global ni el repositorio del proyecto. Las pruebas de integración verifican además que los procesos hijos se cierren y que la raíz generada se elimine de forma controlada.

Host MCP

config/mcp-servers.json declara variables permitidas y servidores stdio o HTTP. La carga es estricta, rechaza campos desconocidos, nombres ambiguos, directorios inseguros, URLs HTTP remotas y sustituciones no declaradas. El host inicia cada cliente, completa el lifecycle, descubre tools y registra el mapping reversible. start_available conserva servidores sanos aunque otro falle, el estado de error permanece visible.

StdioMCPClient administra procesos con shell=False, correlaciona IDs, limita timeouts y termina por EOF. HTTPMCPClient usa urllib, captura la sesión, envía versión y cierra con DELETE. En ambos casos el manager conserva schemas, annotations, structuredContent y isError.

Cada intercambio se añade a un log JSON Lines (JSONL) con categoría, servidor, transporte, dirección, tipo de mensaje, método e ID. La redacción ocurre antes del truncamiento, claves de credenciales y entradas sensibles se sustituyen, datos

binarios y contenido de escrituras se omiten, y existen límites por string y payload. El mensaje real al servidor no se modifica. El cierre recorre todos los clientes aunque uno falle, para no dejar procesos o sesiones abiertos.

Integración con Gemini

Gemini Developer API mediante un POST no streaming a generateContent. El modelo configurado para la validación fue gemini-3.5-flash-lite. La clave se coloca únicamente en el header x-goog-api-key no aparece en URL, cuerpo, logs ni representaciones seguras. La respuesta tiene un límite de 2 MB y el stream HTTP siempre se cierra.

El historial interno usa roles user y assistant, el adaptador los convierte a user y model. Las tools descubiertas se convierten a functionDeclarations. Un functionCall normalizado conserva nombre, argumentos e ID; si el proveedor omite el ID se asigna uno determinista local a la sesión. El functionResponse se correlaciona con la llamada precedente. Si Gemini entrega thoughtSignature, el host la conserva como metadata opaca y la devuelve con la misma llamada futura.

El tool loop tiene límites de rondas, llamadas, texto de entrada, tamaño de resultado e historial. Las llamadas múltiples se ejecutan secuencialmente. Las mutaciones nunca quedan autorizadas por una decisión general, cada operación requiere una confirmación propia y el rechazo se entrega al modelo como resultado de error controlado.

La configuración rechaza modelos y URLs no seguros. El cliente transforma en errores saneados los timeouts, fallos de conexión, HTTP, respuestas demasiado grandes, JSON inválido, candidatos bloqueados o llamadas mal formadas. Los reintentos están deshabilitados por defecto, si se habilitan, son acotados y solo se aplican a fallas transitorias, nunca a errores de autenticación o forma.

La evidencia previa registró un primer turno de búsqueda OTC relacionada con dolor, con acetaminofén e ibuprofeno, un segundo turno que entendió de esos resultados, una consulta `check_stock` para MED-ANA-001 en Zona 5 con stock inicial 25, confirmación visual de `create_order`; creación de una orden ficticia de GTQ 18.95 tras aceptación humana; y stock posterior 24.

Figura 1. *Turno 1 – Búsqueda relacionado con dolor, acetaminofén e ibuprofeno*

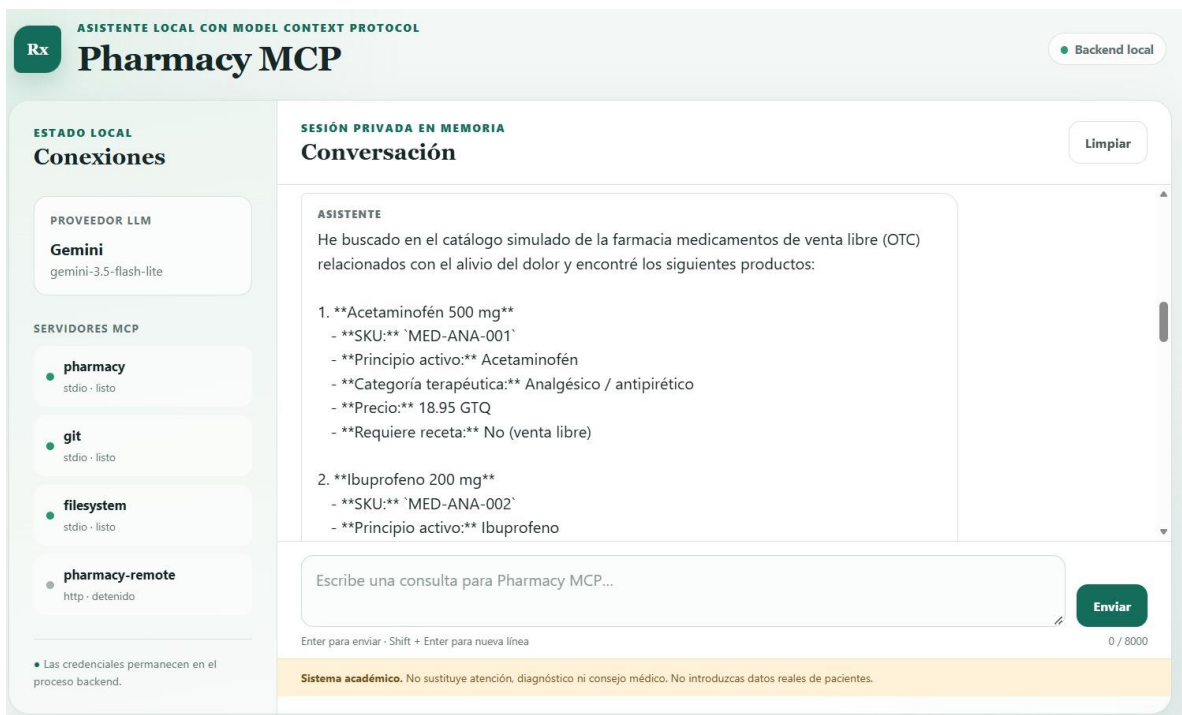
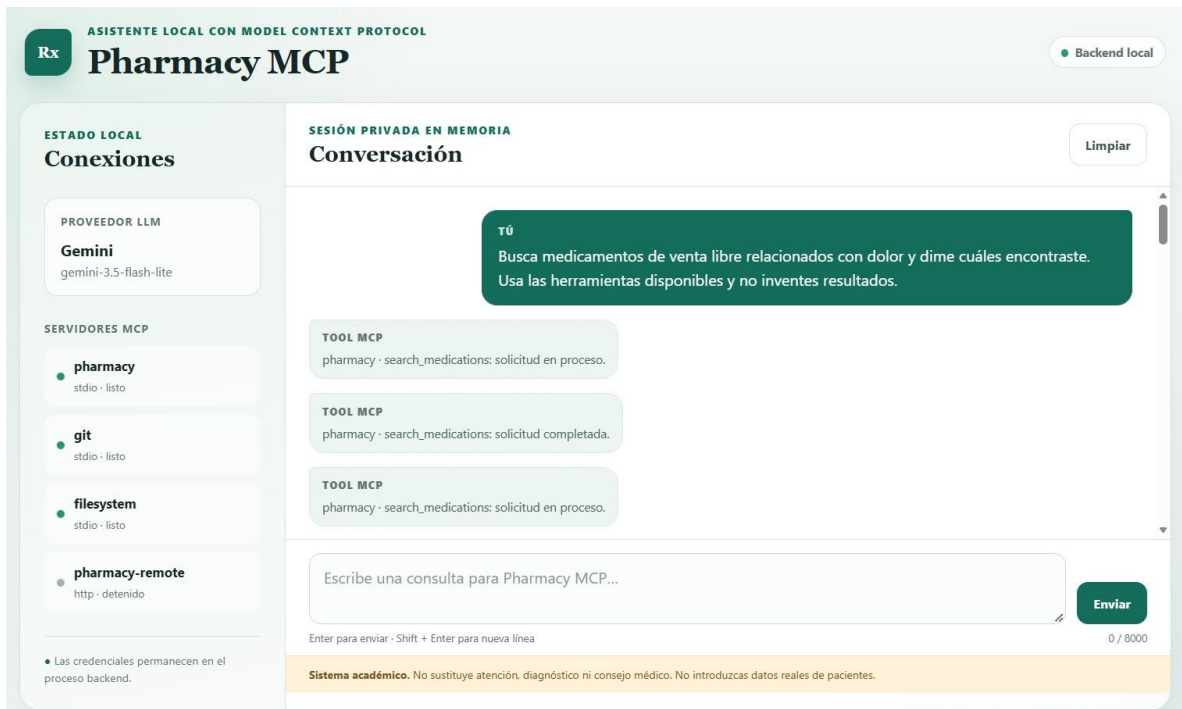


Figura 2. Turno 2 – Uso de contexto y stock

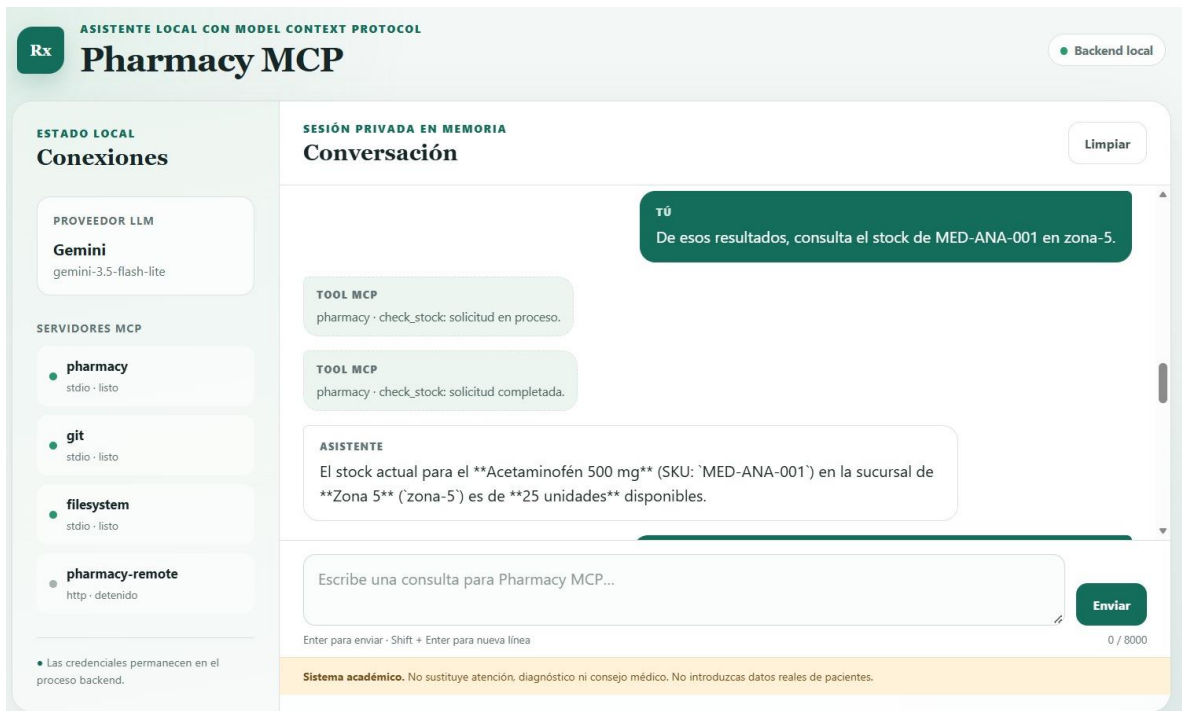


Figura 3. Turno 3 – Creación de orden

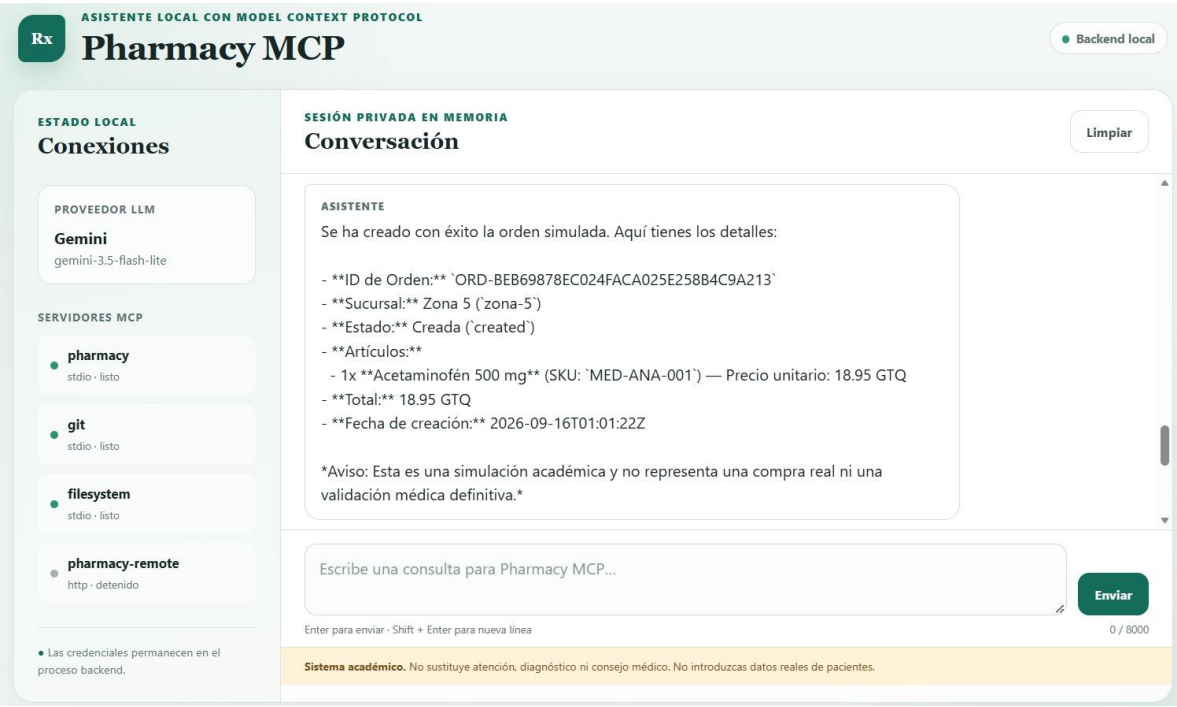
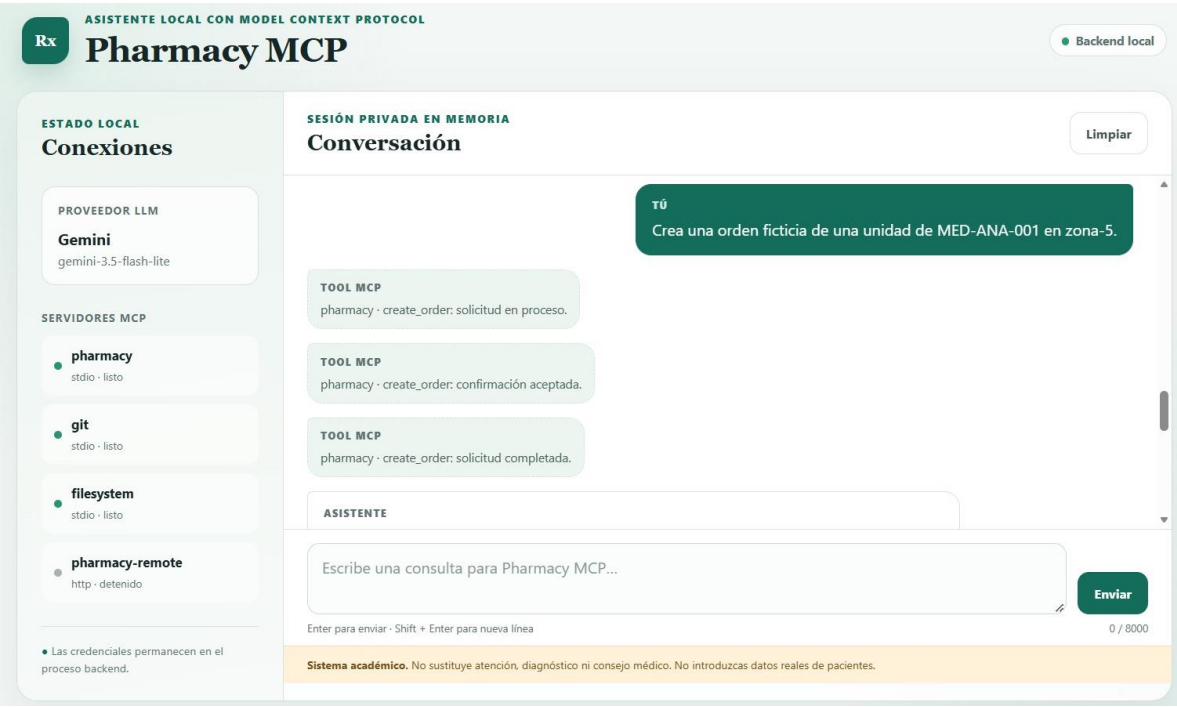
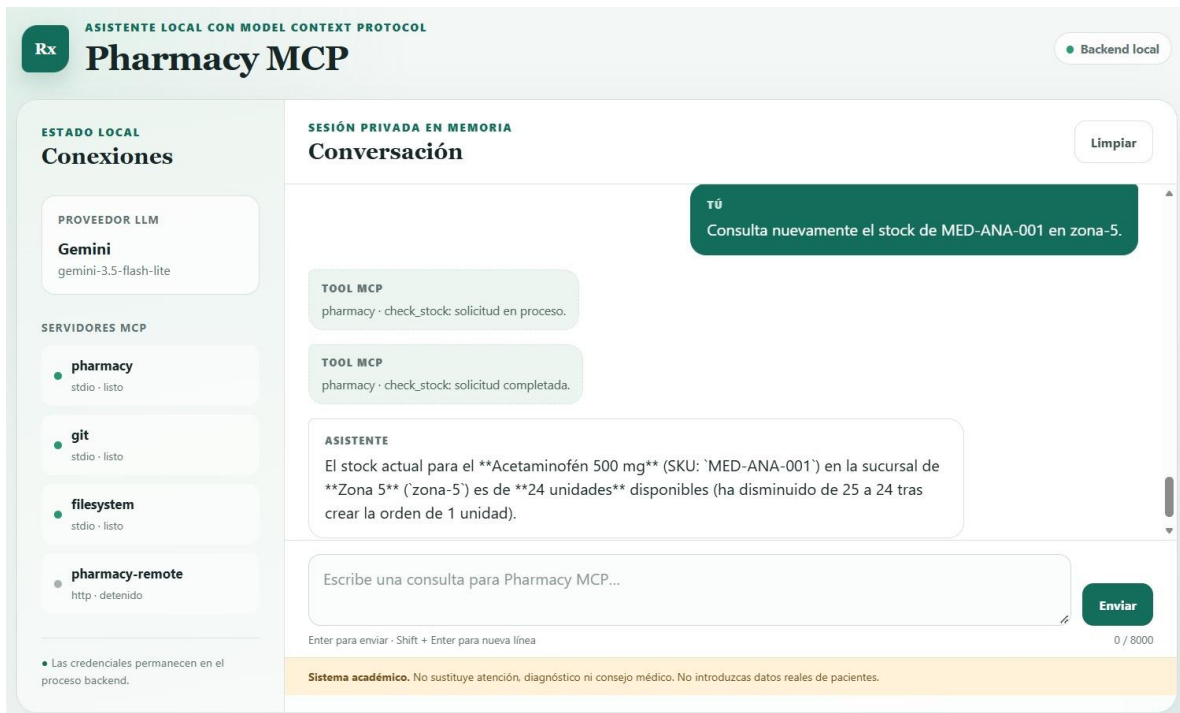


Figura 4. Turno 4 – Disminución de stock



Interfaz Web y Principios HCI

La interfaz es una capa local sobre el mismo ChatOrchestrator, historial, manager y políticas de la CLI. Cada navegador recibe una cookie aleatoria HttpOnly y SameSite=Strict, el backend conserva la conversación en memoria. El frontend consulta una sola vez /api/status al iniciar y utiliza polling de /api/chat únicamente mientras una respuesta o confirmación está pendiente.

La jerarquía visual separa encabezado, estado de proveedor/servidores, conversación, compositor y advertencia médica. Verde y superficies neutras comunican calma y disponibilidad, ámbar y rojo se reservan para advertencias, errores y acciones riesgosas. Los mensajes de usuario, asistente, tool y error se

distinguen por rol. El diseño responsive evita desbordamiento horizontal y mantiene texto y botones legibles en una pantalla aproximada de 390 × 844.

El formulario conserva semántica nativa, Enter envía, Shift+Enter crea una línea; Tab y Shift+Tab recorren controles, Escape rechaza el diálogo. El botón Rechazar recibe foco inicial y ninguna omisión autoriza.

El navegador crea texto con `textContent`, no `innerHTML`, y no usa `localStorage`, `sessionStorage`, CDN ni activos remotos.

Análisis Wireshark

Capa de enlace

En loopback, Wireshark presenta un encabezado Null/Loopback, no hay una trama Ethernet que abandone el equipo. En la captura Wi-Fi remota aparece Ethernet II según la interfaz entregada por Npcap. Las direcciones MAC identifican extremos del enlace local, no al servicio remoto de extremo a extremo; los routers las reemplazan en cada salto.

Capa de red

La comunicación local usa IPv4 de loopback y nunca sale del host. La captura remota seleccionada muestra principalmente IPv6 global, además de flujos observados en el archivo original. El encabezado declara TCP como siguiente protocolo y un hop limit, cada router lo reduce para evitar bucles indefinidos. Las direcciones completas se omiten porque identifican el entorno de captura.

Capa de transporte

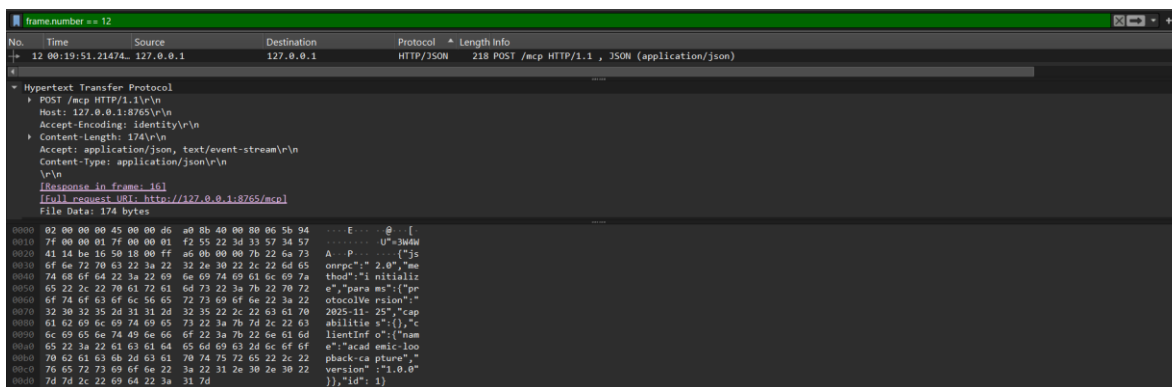
TCP comienza con SYN, continúa con SYN/ACK y se establece con ACK. Los puertos efímeros del cliente se conectan al puerto del servidor local o al 443 remoto. Los números de secuencia y acknowledgements permiten ordenar bytes y detectar pérdidas. La captura local validada muestra FIN/ACK en ambos sentidos y no un RST. En remoto aparecieron varios streams HTTPS, coherentes con solicitudes en conexiones diferentes, su presencia no implica múltiples despliegues.

Capa de aplicación local

Una request incluye id, su response conserva ese valor. La notificación carece de id y solo sincroniza el lifecycle, por lo que HTTP 202 no contiene un resultado JSON-RPC. El identificador opaco de sesión pertenece al transporte HTTP y se envía en mensajes posteriores, pero no se reproduce aquí.

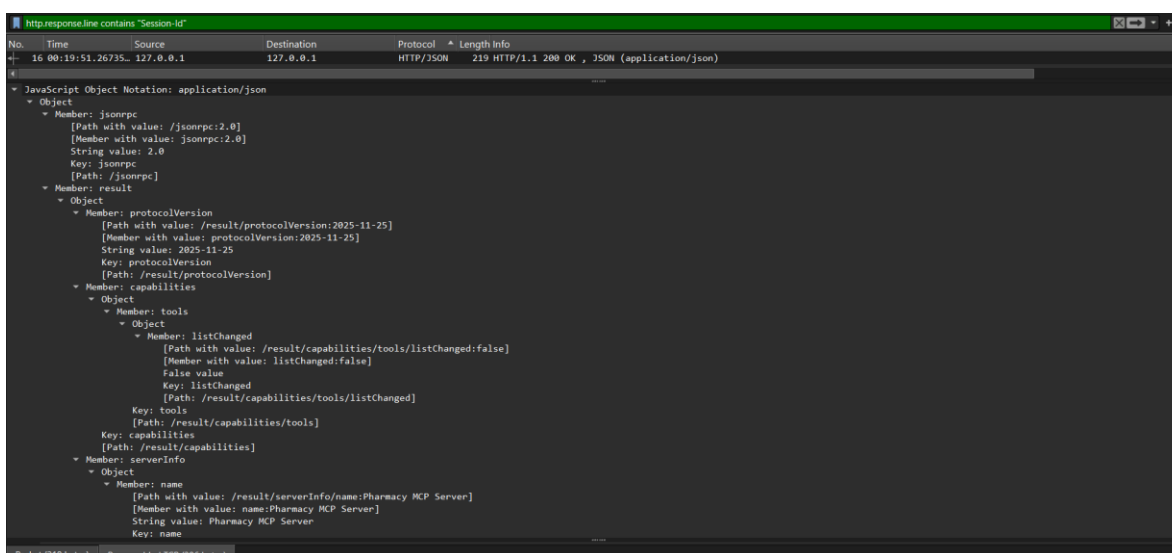
La siguiente captura muestra el envelope HTTP de initialize. Solo aparecen loopback, headers no sensibles y el cuerpo de negociación.

Figura 5. *Solicitud initialize por POST /mcp*



Demuestra la relación entre HTTP y el request JSON-RPC inicial, incluida la revisión MCP.

Figura 6. Resultado JSON-RPC de initialize



El payload visible confirma la versión negociada, la capability de tools y la identidad del servidor.

Figura 7. Respuesta HTTP 202 a notifications/initialized

```
http file_data contains "notifications/initialized" || http.response.code == 202
No. Time Source Destination Protocol Length Info
-- 22 00:19:51.26858.. 127.0.0.1 127.0.0.1 HTTP 176 HTTP/1.1 202 Accepted
20 00:19:51.26794.. 127.0.0.1 127.0.0.1 HTTP/JSON 110 POST /mcp HTTP/1.1 , JSON (application/json)

Frame 22: Packet, 176 bytes on wire (1408 bits), 176 bytes captured (1408 bits) on interface \Device\NPF_{...} id 0
Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 8765, Dst Port: 62037, Seq: 571, Ack: 772, Len: 132
Hypertext Transfer Protocol
  HTTP/1.1 202 Accepted\r\n
    Response Version: HTTP/1.1
    Status Code: 202
    [Status Code Description: Accepted]
    Response Phrase: Accepted
    Server: PharmacyMCPHTTP \r\n\r\n
    Date: Sat, 12 Sep 2026 06:19:51 GMT\r\n
    Cache-Control: no-store\r\n\r\n
    Content-Length: 0\r\n
    [Content length: 0]
  \r\n\r\n
  [Request in frame: 20]
  [Time since request: 645.200 microseconds]
  [Request URI: /mcp]
  [Full request URI: http://127.0.0.1:8765/mcp]

0000 02 00 00 00 45 00 00 ac a0 95 40 00 80 06 5b b4 ... E... .0... [
0010 7f 00 00 01 7f 00 00 01 22 3d f2 55 41 14 bf a2 ..... "UA...
0020 33 57 36 4e 50 18 00 fc 7a 16 00 00 48 54 54 50 300MP... z... HTTP
0030 2f 31 2a 31 20 32 30 32 20 41 63 65 70 74 65 /1.1 202 Accepta
0040 64 0d 0a 53 65 72 76 65 72 3a 20 50 68 61 72 6d d Serve r: Pharm
0050 61 63 79 4d 43 50 48 54 54 50 20 0d 0a 44 61 74 acyMCPHT TP : Dat
0060 65 3a 20 53 61 74 2c 20 31 32 20 53 65 70 20 32 e: Sat, 12 Sep 2
0070 30 32 36 20 30 36 3a 31 39 3a 35 31 20 47 4d 54 026 06:1 9:51 GMT
0080 0d 0a 43 61 63 68 65 2d 43 6f 6e 74 72 6f 6c 3a . Cache- Control:
0090 20 6e 6f 2d 73 74 6f 72 65 0d 0a 43 6f 6e 74 65 no-stor e Conte
00a0 6e 74 2d 4c 65 6e 6f 74 68 3a 20 30 0d 0a 0d 0a nt-Lengt h: 0
```

El cuerpo tiene longitud cero, lo que corresponde a una notificación sin respuesta JSON-RPC.

Dificultades y soluciones

Dificultad	Causa	Solución	Aprendizaje
Error JSON-RPC frente a error de dominio	No toda falla de tool implica un request inválido	-32602 para forma inválida; result.isError para fallas esperables del dominio	Separar contrato de ejecución mejora interoperabilidad
Atomicidad SQLite	Una orden de varias líneas podía quedar parcial	BEGIN IMMEDIATE, actualización condicionada,	La transacción debe abarcar validación, reserva e inserción

		constraints y rollback	
Concurrencia	Dos escritores podían competir por las últimas unidades	Reserva de escritura y quantity >= requested en el UPDATE	Validar y actualizar deben ocurrir dentro del mismo control transaccional
Streamable HTTP	Lifecycle, sesión y HTTP tienen errores distintos	Sesiones aisladas, headers de versión, 202 para notifications y DELETE	Transporte y protocolo deben validarse en capas distintas

Lecciones Aprendidas

La separación entre protocolo, dominio, transporte, host, proveedor y UI hizo posible probar cada nivel y reutilizar el mismo servidor por stdio y HTTP. JSON-RPC y MCP mostraron que la interoperabilidad surge de contratos precisos, no de acoplar herramientas a un modelo específico.

Wireshark permitió relacionar abstracciones de aplicación con TCP, IP, enlace y TLS, y aclaró por qué el cifrado impide atribuir contenido a un registro remoto aislado.

La nube añadió restricciones de imagen, identidad, costo y persistencia, la UI añadió claridad, responsive design y accesibilidad.

Conclusiones

1. El host demostró interoperabilidad al administrar Pharmacy, Git y Filesystem, aplicar namespaces y sostener servidores disponibles ante fallos parciales.
2. Gemini pudo mantener contexto y coordinar tools mediante REST, con confirmación humana antes de una mutación.
3. SQLite ofreció atomicidad y concurrencia suficientes para una demostración de una instancia, pero no durabilidad de nube ni coordinación multi-instancia.
4. Las pruebas automatizadas, validaciones manuales y capturas Wireshark cubren aspectos complementarios, ninguna fuente individual demuestra todo el sistema.

Repositorio

El código realizado para los avances 1 del Chatbot se encuentran en el [link](#) (Click en la imagen):

REPOSITORIO

GITHUB



Referencias

- Docker. (s. f.). Docker Build documentation. Docker Documentation.
<https://docs.docker.com/build/>
- Google. (s. f.-a). Gemini API: Text generation. Google AI for Developers.
<https://ai.google.dev/gemini-api/docs/generate-content/text-generation>
- Google. (s. f.-b). Gemini API: Function calling. Google AI for Developers.
<https://ai.google.dev/gemini-api/docs/function-calling>
- Google Cloud. (s. f.-a). Cloud Run container runtime contract. Google Cloud. <https://cloud.google.com/run/docs/container-contract>
- JSON-RPC Working Group. (s. f.). JSON-RPC 2.0 specification. JSON-RPC. <https://www.jsonrpc.org/specification>
- Model Context Protocol. (2025a). MCP specification.
<https://modelcontextprotocol.io/specification/2025-11-25>
- Model Context Protocol. (2025b). Architecture.
<https://modelcontextprotocol.io/specification/2025-11-25/architecture>
- Model Context Protocol. (2025c). Lifecycle.
<https://modelcontextprotocol.io/specification/2025-11-25/basic/lifecycle>

- Model Context Protocol. (2025d). Transports.
<https://modelcontextprotocol.io/specification/2025-11-25/basic/transports>
- Model Context Protocol. (2025e). Tools.
<https://modelcontextprotocol.io/specification/2025-11-25/server/tools>
- Model Context Protocol. (s. f.-a). Servers: Official repository . GitHub.
<https://github.com/modelcontextprotocol/servers>
- Model Context Protocol. (s. f.-b). Git server. GitHub.
<https://github.com/modelcontextprotocol/servers/tree/main/src/git>
- Model Context Protocol. (s. f.-c). Filesystem server. GitHub.
<https://github.com/modelcontextprotocol/servers/tree/main/src/filesystem>
- Model Context Protocol. (2026). mcp-server-git (versión 2026.8.18). PyPI.
<https://pypi.org/project/mcp-server-git/2026.8.18/>